# JosephsonCircuitsOptimizer.jl

## Quick Start

Design, optimization and nonlinear analysis of Josephson circuits

| MODEL | LINEAR | OPTIMIZE | HB | FEEDBACK |
|---|---|---|---|---|

JosephsonCircuitsOptimizer.jl (JCO) organizes simulations from JosephsonCircuits.jl (https://josephsoncircuits.org/) into linear exploration, surrogate optimization, and nonlinear harmonic-balance (HB) analysis. This guide focuses on the controls and functions needed to run, understand, and preserve an experiment.

## 1. Quick start

- 1. Install Julia and download the JCO desktop package for Windows or Linux from GitHub Releases (https://github.com/SQE-INRiM/JosephsonCircuitsOptimizer/releases).
- 2. Open JCO, then use Julia settings to select the Julia executable and number of threads.
- 3. Open a bundled example or create a project. Define the circuit, parameters, sources and metrics under Setup.
- 4. Save the project as a .jco file, run the computation, and inspect the numerical data under Results.

The first launch may require internet access while the Julia environment is prepared. Windows offers an installer and a portable executable. Linux uses an x86-64 AppImage; Linux packaged support is preliminary and has been less extensively tested than Windows.

## 2. Desktop GUI

The interface has three main pages:

- Setup - Configure the experiment through Model & inputs, Computation, and Metrics & constraints. A circuit preview can display the resolved lumped-element model.
- Run - Start the complete workflow or an available individual stage. Monitor Linear, Optimization, HB, and (when enabled) feedback-cycle progress, logs and termination status.
- Results - Select a stored run, inspect Linear/Optimization/HB datasets, compare up to two points, and export CSV data. Feedback runs also provide cycle selection and a convergence overview.

A .jco project may contain several runs. The Run selector in Results selects a saved experiment independently of the configuration currently being edited.

## 3. Model and sources

### Circuit and device parameters

Edit the Julia circuit in Model & inputs -> Circuit. Its function receives a dictionary of parameters and returns a JosephsonCircuits circuit and its definitions:

```
function create_user_circuit(device_params_set)
    # Define components, nodes, and ports.
    return circuit, circuitdefs
end
```

Under Device parameters, give each parameter a name, unit, and a mode:

- Fixed - One value; no design-space sweep along this parameter.
- Range - Start, stop and step; regular design-space sampling.
- List - Explicit comma-separated values.

The names used in this table must match the keys accessed by device_params_set in the circuit code. Use SI values in the Julia model; the GUI displays the selected units.

### AC, DC and parametric sources

In Sources, add a port and choose AC or DC. AC sources have a frequency setting; DC sources use zero frequency. Set the linear and nonlinear amplitudes separately. Fixed, Range and List are available; a Parametric source can call a user Julia function of device_params_set.

The GUI can display supported AC amplitudes in A or dBm. The Julia solver receives physical source currents in amperes; DC currents and feedback reference currents are specified in A. Check the source impedance used for dBm conversion. Switching an AC source to DC and back preserves its AC settings in the editor.

The Linear sweep uses the low-power source amplitudes. The Nonlinear stage explores the configured nonlinear amplitudes and source frequencies for the selected device.

## 4. Computation and run modes

### Linear sweep and optimization

Under Computation -> Linear sweep, choose the probe-frequency Start and Stop (GHz) and Step (MHz). JCO evaluates the user-defined cost at the uniform device-parameter grid.

Optimization uses the evaluated linear dataset to initialize a surrogate and sample further points. Configure Iterations, Samples / iteration, Random seed, Surrogate, Strategy, and Sampler. The GUI currently offers Kriging, radial basis and second-order polynomial surrogates; SRBF, expected improvement and lower confidence bound strategies; and random, Sobol or Latin-hypercube sampling.

The first value returned by user_cost is minimized. Masks and physical constraints are defined by your Julia cost function; JCO does not infer them from plot selections. If there is only one device configuration, optimization is skipped. The current Julia optimization backend expects at least two parameter columns.

### Harmonic-balance settings

Harmonic and solver settings separates linear and nonlinear strong-tone/modulation harmonic limits. A conventional low-power linear sweep uses first-order harmonic limits; choose sufficient nonlinear harmonics for the driven system.

Shared hbsolve settings include maximum iterations, ftol, line-search controls, intermodulation order, frequency batches and node sorting. Three-wave and four-wave mixing are explicit options. Increasing harmonics or the size of the circuit can substantially raise computation time and memory use.

### Starting a run

Run executes Linear -> Optimization (when applicable) -> Harmonic balance. Stage-only actions allow reuse of compatible saved inputs/results, but they do not execute feedback cycles. Feedback is part of a full run with at least one varying design parameter. A single fixed-device configuration can still be analyzed with HB.

A running simulation can be stopped; interrupted or non-converged results should not be treated as completed scientific data. JCO retains the run information and numerical results that were successfully saved.

## 5. Custom metrics and saved data

In Metrics & constraints edit three Julia functions independently: Cost, Performance, and Nonlinear correction. The editors preserve other functions and helpers in the shared user_cost_and_performance.jl file; use the complete-file editor when needed.

```
function user_cost(sol, device_params_set, nonlinear_correction)
    s11 = sol.linearized.S((0,), 1, (0,), 1, :)
    s11_db = 10 .* log10.(abs2.(s11))
    objective = mean(s11_db) + nonlinear_correction
    return (metric = objective, mean_s11_db = mean(s11_db))
end
```

```
function user_performance(sol, device_params_set, source_amps, source_freqs)
    s21 = sol.linearized.S((0,), 2, (0,), 1, :)
    gain_db = 10 .* log10.(abs2.(s21))
    return (performance = mean(gain_db), peak_gain_db = maximum(gain_db))
end
```

In both functions, sol is the complete JosephsonCircuits HB solution, not an S-parameter dictionary. The first named returned value is the scalar objective: cost is minimized, performance is maximized. Additional named scalar values are stored for analysis. The S accessor shown above extracts a fundamental-frequency, small-signal linearized response; ports 1 and 2 are examples and must match your circuit.

To implement masks, evaluate conditions inside user_cost and assign a penalized metric when the design is invalid. Return useful mask diagnostics as additional named fields if you want them visible in Results. Verify that the optimization objective and penalty convention are consistent.

For frequency-dependent arrays, use @save_data variable_name in user code. With the default selected-data retention, only explicitly saved arrays are available for point-resolved plots; summary metrics are stored independently. See Section 8 for the advanced trace modes.

# 6. Nonlinear feedback

Nonlinear feedback is optional. It repeats the design-space exploration and optimization using a user-defined scalar correction calculated from low-power and pumped HB solutions. It is configured in Computation -> Nonlinear feedback and is only executed as part of a full, multi-configuration run.

## Define the correction

Implement user_nonlinear_correction(sol_linear, sol_nonlinear, device_params_set). Both inputs are full HB solutions. For example, a correction based on the difference in average reflected power response is:

```
function user_nonlinear_correction(sol_linear, sol_nonlinear, device_params_set)
    s_lin = sol_linear.linearized.S((0,), 1, (0,), 1, :)
    s_on  = sol_nonlinear.linearized.S((0,), 1, (0,), 1, :)
    db(x) = 10 .* log10.(abs2.(x))
    return mean(db(s_on)) - mean(db(s_lin))
end
```

JCO passes the returned value as nonlinear_correction to user_cost in the next corrected cycle. Your cost function must explicitly use the value; otherwise enabling feedback does not change the objective. The meaning and units of the correction are defined by your Julia code.

> Important: a correction applied as the same constant offset to every candidate does not, by itself, move the minimizer. Feedback convergence alone is not evidence of improved gain or a physically better design.

## Configure reference working points

Enable nonlinear feedback and enter one reference current (A) for each source, including DC sources. These are the operating currents used to evaluate the correction at the optimized device; they do not change the nonlinear sweep grid.

If the preceding HB sweep already contains a converged point with the same complete source-current and source-frequency vectors, JCO reuses its calculated correction. Otherwise, it performs an additional linear and nonlinear solve at the reference point.

## Choose when to stop

Choose exactly one stopping mode:

- Fixed cycles - Run the specified number of corrected iterations.
- Metric convergence - Select Monitor (Cost, Performance, or Correction), Change (absolute or relative), Tolerance, and Maximum cycles (at least two).

The monitored values are, respectively, the optimized cost (minimized), the best evaluated HB performance in that cycle (maximized), or the user correction. For successive completed corrected cycles with values a and b, JCO stops when:

```
Absolute:  abs(b - a) <= tolerance
Relative:  100 * abs(b - a) / max(abs(a), abs(b), eps) <= tolerance (%)
```

Absolute tolerance uses the units of the selected metric; relative tolerance is a percentage. Cycle 0 is the uncorrected baseline and is not counted as a corrected iteration or used as the first convergence comparison. An upper cycle limit remains in force even if convergence is not reached.

### Read feedback results

During Run, the stage cards show each cycle's Linear, Optimization and HB progress, and a separate feedback indicator counts completed corrected cycles. In Results, select Cycle 0 or a corrected cycle in the strip above the stage tabs. The selected cycle controls the displayed numerical datasets, stored traces, comparisons and CSV exports.

The Nonlinear feedback view summarizes the best evaluated HB performance and the saved parameter history across completed cycles. A reported convergence status means only that the chosen stopping rule was met; verify physical quality and numerical HB convergence separately.

## 7. Results and exports

- Linear: Inspect the cost/metric heatmap, slices through additional parameters, selected points and retained frequency-dependent arrays.
- Optimization: Inspect the evaluated landscape (projection or aligned Linear slice) and the objective History. A history curve is not shown when fewer than two eligible evaluations exist.
- Harmonic balance: Inspect performance over nonlinear working points, amplitudes/frequencies, convergence and retained arrays. A single working point does not provide an amplitude-response curve.
- Point comparison: Select up to two points within the same selected run/cycle; cross-cycle overlay is not supported.
- Export: Use Export CSV for a numerical stage or the plot export controls for selected curves. Exporting is independent of saving the .jco project.

Simulation info shows saved run settings. Manage runs or Delete run operates on stored runs, not on the active Setup fields. Deletion of a run removes its stored results and cannot be undone.

## 8. Projects, files and reproducibility

A .jco is a portable, versioned ZIP project containing editable inputs, helper files and the preserved numerical runs. Save/Save As creates or updates the archive; Open and drag-and-drop can reopen it. Each new run is stored separately, so previous results remain available.

The required files within the project workspace are:

- device_parameters_space.json - fixed, range and list design parameters.
- drive_physical_quantities.json - probe frequencies, source ports/currents/frequencies and reference currents.
- simulation_config.json - harmonics, hbsolve, data retention and feedback settings.
- optimizer_config.json - surrogate, strategy, sampling and iteration settings.
- user_circuit.jl - circuit definition and component values.
- user_cost_and_performance.jl - cost, performance, correction and helper functions.

Numerical summaries are stored in HDF5 files for Linear, Optimization and HB, with point identifiers linking rows to saved arrays. With trace_storage_mode = "selected" (default), explicitly saved arrays are retained. "all" also retains full fundamental S-parameter diagnostic traces, increasing storage; "none" keeps summaries without detailed arrays. The GUI derives plots from stored numerical data rather than relying on exported images.

Advanced feedback settings in simulation_config.json are n_iterations_nonlinear_correction, nonlinear_feedback_stop_mode, nonlinear_feedback_metric, nonlinear_feedback_tolerance_type, and nonlinear_feedback_tolerance. The reference currents are stored under source_<i>_non_linear_amplitude_for_delta_correction in the source configuration. Existing projects without the newer stopping keys default to fixed cycles.

## 9. Practical checks

Before trusting a result, verify the port convention and source units; check S-parameter indices, mask behavior and frequency ranges; verify solver convergence at the chosen amplitudes; and confirm that the saved run/cycle corresponds to the settings you intend to report.

On Windows, unsigned research builds may trigger SmartScreen. Windows 11 Smart App Control can separately block Julia native libraries; see the Microsoft FAQ (https://support.microsoft.com/en-us/windows/smart-app-control-faq-285ea03d-fa88-4a37-8837-aacbe6d2e7d0). Disabling a security feature lowers protection and is not a general installation recommendation.

> Linux support remains preliminary: packaged support is x86-64, has been less extensively validated than Windows, and may depend on distribution-specific libraries.

## 10. References


- Repository and issues: https://github.com/SQE-INRiM/JosephsonCircuitsOptimizer
- Circuit simulator: https://josephsoncircuits.org/
- Project format details: docs/PROJECT_FORMAT.md in the repository.
- Citation: E. Palumbo et al., "JCO: Optimization Framework for Nonlinear Superconducting Circuits Using a Lumped-Element Approach and Harmonic Balance," IEEE Transactions on Applied Superconductivity (2026), DOI: 10.1109/TASC.2026.3679488 (https://doi.org/10.1109/TASC.2026.3679488).